

DOCUMENTO DE ESPECIFICACIÓN Y PLANEACIÓN

Curso: Programación en JavaScript y TypeScript

Fase de Desarrollo: 1 - Pensamiento Sistémico e Ingeniería de Requisitos

1. Caracterización del Sistema

Para comprender la naturaleza y el alcance de **UniBot**, se define el sistema bajo la teoría general de sistemas:

- **Objetivo:** Proveer un sistema de asistencia automatizada y acotada académicamente para estudiantes de primer ingreso mediante una aplicación de consola en Node.js, optimizando el tiempo de los asesores humanos y evitando la propagación de información errónea sobre planes de estudio ajenos.
- **Entradas:**
 1. Credenciales de acceso ingresadas por consola (código institucional del estudiante o contraseña maestra del administrador).
 2. Archivos locales de persistencia estructurados (`estudiantes.csv` y `carreras.json`).
 3. Consultas, preguntas y comandos de salida escritos por el estudiante durante la sesión de chat.
 4. Comandos administrativos de auditoría y ejecución de cierres forzados de sesiones.
- **Procesos:**
 1. Validación de la identidad del usuario mediante la lectura y búsqueda en archivos planos.
 2. Construcción dinámica de restricciones contextuales a través de la clase `PromptBuilder`.
 3. Registro y actualización del ciclo de vida de las sesiones (`registro_sesiones.json`), incluyendo el conteo de caracteres y el cálculo de costos operativos en COP.
 4. Gestión de peticiones asíncronas hacia la API externa del LLM.
 5. Generación de trazas históricas de las conversaciones en archivos de texto plano dentro de la carpeta `/logs`.
- **Salidas:**
 1. Mensajes de respuesta del asistente virtual acotados exclusivamente a la temática de la carrera del estudiante.
 2. Paneles de reporte y auditoría de sesiones activas/cerradas en la consola para el administrador.
 3. Archivos físicos de bitácora por cada sesión finalizada.
- **Elementos (Subsistemas Involucrados):**
 1. *Subsistema de Interfaz de Usuario:* Gestión de menús duales en consola mediante asincronismo (`readline/promises`).
 2. *Subsistema de Persistencia Local:* Manipulación segura del sistema de archivos mediante módulos nativos (`fs` y `path`).
 3. *Subsistema de Inteligencia Artificial:* Enrutamiento de prompts y comunicación con servicios de red externos (`LLMService` y `PromptBuilder`).

4. *Subsistema de Control y Auditoría*: Monitoreo de estados, conteo de caracteres y tarificación de costos.

2. Modelo de Proceso y Justificación del Marco de Trabajo

Para el desarrollo del proyecto **UniBot**, se ha seleccionado un **Modelo de Desarrollo Incremental** bajo un enfoque adaptado de **Scrum**.

Justificación:

La complejidad del software evoluciona de manera escalonada: parte de un análisis conceptual, avanza hacia un prototipo estructurado basado en archivos locales planos (JavaScript), migra hacia una arquitectura robusta orientada a objetos con tipado estricto (TypeScript) y culmina con la integración de APIs asíncronas e inteligencia artificial. El enfoque ágil e incremental permite mitigar riesgos técnicos en fases tempranas, validar la lógica de negocio por bloques funcionales (Sprints) y asegurar entregas de valor continuo alineadas con los requerimientos institucionales.

Organización por Sprints:

- **Sprint 1 (Fase 1 - Actual)**: Pensamiento sistémico, especificación de requisitos, modelado UML y estructuración del tablero de control (Kanban).
- **Sprint 2 (Fase 2)**: Creación del prototipo estructurado en JavaScript (`index.js`), implementación del menú dual, configuración de variables de entorno (`.env`) y lectura del archivo `estudiantes.csv`.
- **Sprint 3 (Fase 3)**: Refactorización completa a TypeScript (`tsconfig.json`), desarrollo de clases orientadas a objetos (`LectorCSV`, `LectorJSON`, `Estudiante`, `Carrera`), implementación de escritura en CSV y modificación dinámica de reglas JSON.
- **Sprint 4 (Fase 4)**: Implementación del módulo de interacción con el LLM, desarrollo de la clase `PromptBuilder`, sistema de auditoría de sesiones con cálculo de costos en COP, gestión robusta de errores (`try/catch`) y almacenamiento de logs en `/logs`.

3. Especificación de Requerimientos (Product Backlog Inicial)

El siguiente Product Backlog detalla los requerimientos funcionales (RF) y no funcionales (RNF) priorizados para la construcción iterativa del sistema:

ID	Tipo	Descripción de la Historia / Requerimiento	Prioridad
RF-01	Funcional	Diseñar un menú dual interactivo en consola que permita seleccionar entre el rol de Administrador y Estudiante.	Alta

RF-02	Funcional	Implementar la autenticación segura del Administrador utilizando una contraseña maestra consumida desde un archivo <code>.env</code> .	Alta
RF-03	Funcional	Desarrollar la función de lectura y validación del archivo <code>estudiantes.csv</code> para verificar el código institucional de ingreso del estudiante.	Alta
RF-04	Funcional	Habilitar al Administrador la opción de registrar nuevos estudiantes agregando líneas de texto al archivo CSV (<code>fs.appendFileSync</code>).	Media
RF-05	Funcional	Permitir al Administrador modificar dinámicamente las reglas temáticas de <code>carreras.json</code> agregando nuevos temas permitidos.	Media
RF-06	Funcional	Implementar el registro de sesiones en <code>registro_sesiones.json</code> contemplando conteo de mensajes, caracteres y cálculo de costos en COP.	Alta
RF-07	Funcional	Incorporar la función de auditoría y cierre forzado de sesiones activas en el panel del Administrador.	Media
RF-08	Funcional	Construir la clase <code>PromptBuilder</code> para encapsular las restricciones institucionales, el nombre, la carrera y los temas permitidos del estudiante.	Alta

RF-09	Funcional	Integrar el servicio asíncrono del LLM y asegurar la exportación del historial de chat a un archivo <code>.txt</code> en la carpeta <code>/logs</code> al salir.	Alta
RNF-01	No Funcional	Migrar y estructurar todo el código fuente bajo tipado estricto utilizando TypeScript (Fases 3 y 4).	Alta
RNF-02	No Funcional	Garantizar la robustez del sistema mediante bloques globales y específicos <code>try/catch</code> para prevenir caídas (<i>crashes</i>) por fallas de archivos o red.	Alta
RNF-03	No Funcional	Utilizar idioma inglés en los identificadores del código fuente, aplicando convenciones estándar (<i>CamelCase</i> para variables/métodos y <i>PascalCase</i> para clases).	Media

4. Modelado Estructural

(Nota: Puedes representar este modelo gráficamente en herramientas como Draw.io, Lucidchart o código Mermaid e insertar la imagen resultante en tu PDF).

Actores del Sistema:

1. **Administrador del Sistema (Back-Office):** Usuario con privilegios elevados encargado de la gestión de accesos, actualización de reglas temáticas y auditoría de sesiones.
2. **Estudiante (Front-Office):** Usuario regular que accede mediante su código institucional para realizar consultas académicas supervisadas.

Casos de Uso Identificados:

- **CU-01: Autenticar Administrador** (Valida contraseña maestra desde `.env`).
- **CU-02: Gestionar Estudiantes** (Agrega registros al archivo `estudiantes.csv`).
- **CU-03: Gestionar Reglas Académicas** (Actualiza temas permitidos en `carreras.json`).
- **CU-04: Auditar y Cerrar Sesiones** (Monitorea estados y ejecuta cierres forzados con cálculo de costos).
- **CU-05: Autenticar Estudiante** (Valida código institucional contra el archivo CSV).
- **CU-06: Interactuar con el Asistente IA** (Envía preguntas protegidas por el `PromptBuilder`).
- **CU-07: Finalizar Sesión y Generar Bitácora** (Guarda el historial completo de conversación en un archivo `.txt` bajo la carpeta `/logs`).